

HealthConnect

A simple booking and medical report parsing platform

- Domain: Healthcare & Digital Medical Records
- Core Stack: React, Node.js, Express, PostgreSQL, Python Flask, Cloudinary CDN
- Main Impact: Speeds up booking and automatically triages patients based on report scans.

The Problems We Are Solving

Why traditional appointment systems are difficult to build and run:

Double Booking Slots

Two users click the same appointment slot simultaneously. Without proper locks on the database level, both get booked, leading to schedule conflicts.

Main Server Slowdown

Reading text from files (OCR) takes a lot of CPU power. Running this directly on the main Node server freezes it and delays requests for all other users.

Running Out of Disk Space

Storing large medical reports and images directly on the application servers fills up disk storage quickly and slows down server performance.

System Requirements

What the System Does

- Patients: Scan reports, search specialists, book slots, and pay.
- Doctors: Set slot availability, approve bookings, write prescriptions.
- Pharmacy: Receive order notifications, fulfill medicines, view wallet earnings.

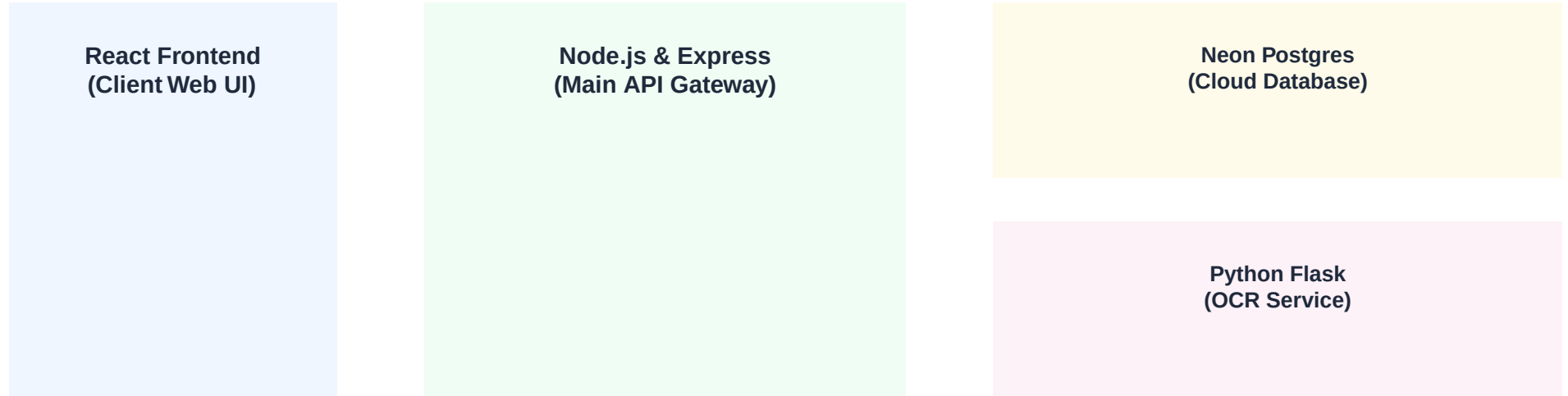
How the System Behaves

- Zero Double-Booking: Locks the slot instantly while payment is processing.
- High Availability: Heavy text extraction runs on a separate backend container.
- Transaction Safety: Holds patient payment in escrow. Payout occurs only on prescription.

Technology Stack Selection

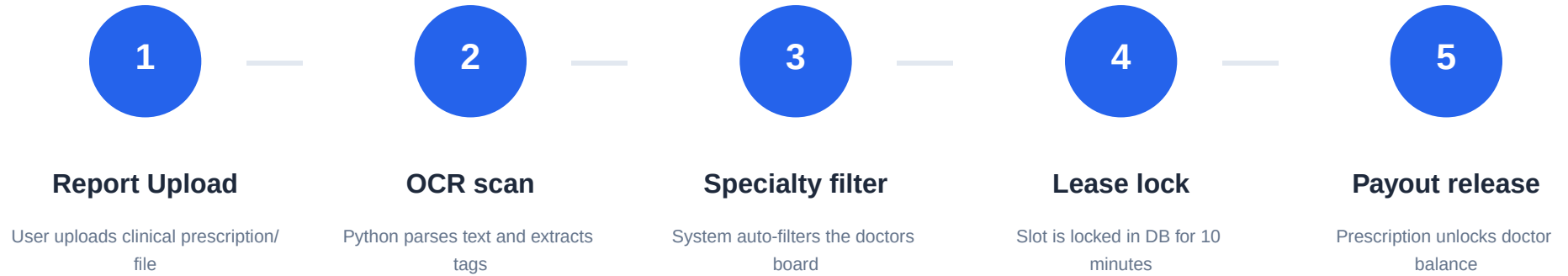
Layer	Choice	Justification
Frontend UI	React	Builds highly interactive layouts easily. Renders data fast for responsive user dashboards.
Main Server API	Node.js & Express	Handles many concurrent users at once because of its fast, non-blocking asynchronous event handler.
Database	PostgreSQL (Neon)	Relational database with strict rules (ACID) to store clinic balances and slots safely.
Text Scan Service	Python Flask	Separate OCR service so the main server stays fast and does not block public requests.
File Cloud CDN	Cloudinary	Stores user medical files in the cloud, saving local container disk space and removing storage limits.

High-Level Architecture



- Request Pipeline: React client uploads a report -> Node forwards it to Python Flask OCR -> Flask returns triage department.
- Decoupled Isolation: Flask OCR is isolated. Even if OCR gets heavy traffic, patients can still browse slots without lag.

System Data Flow Lifecycle



Feature Matrix Modules

Patient Portal

- Scan medical reports
- Search & filter doctors
- Book slots with locking
- Order medicines
- Live Doctor Chat

Doctor Portal

- Add/Delete open slots
- Review booking cards
- Accept or reject requests
- Prescribe medicines
- Check earnings wallet

Merchant / Admin

- Fulfill patient drug orders
- Manage inventory listings
- Audit balances & commission
- Approve newly registered practitioners

Database Table Schemas

Relational database schema enforcing reference integrity across system tables:

users (Patients)

Columns: id, fullName, username, password, email, mobile, gender, age

doctors

Columns: id, fullName, username, password, email, specialization, slots (JSON), balance

vendors

Columns: id, fullName, username, storeName, inventory (JSON), balance

appointments

Columns: id, patientUsername, doctorUsername, specialization, slot, symptoms, paymentStatus, escrowStatus

Low-Level Directory Layout

Routes Module

publicRoutes.js, appointmentRoutes.js

Sets API endpoints and connects route paths to controllers while managing authorization.

Controllers

authController.js, appointmentController.js

Verifies user request parameters and formats success/error JSON payloads.

Services

userService.js, otpService.js, emailService.js

Performs system business validation, hashes keys, and manages OTP sessions.

Repositories

doctorRepository.js, appointmentRepository.js

Writes clean SQL transactions. Keeps database logic isolated for easy testing.

Engineering Challenges Faced

Engineering Challenge	Why It Happened	How I Solved It
PostgreSQL Lowercase Key Bug	PostgreSQL changes column names returned in queries to all-lowercase, breaking camelCase keys in React.	Built a central mapping interceptor in dbHelpers.js to normalize returning rows back to camelCase before sending them.
Monolithic Latency / Slow Node	Text scanning (OCR) takes high CPU processing. Inline execution frozen main server API threads.	Decoupled the text extractor into a separate Python Flask microservice, keeping Node.js completely responsive.
Blocked CORS Domain Access	Deploying clients on Vercel and gateways on Railway caused browser security blocks.	Applied dynamic Express CORS middleware configurations to authorize custom client headers.

Scalability Roadmap

- Redis Cache-Aside Pattern:

Add an in-memory Redis cache for doctors list and schedules. Reduces database query loads by up to 80%.

- Asynchronous Task Queues:

Move OCR uploads to background queues (like BullMQ) using Redis, letting users book slots immediately.

- Container Containment:

Package services inside Docker containers for reliable scaling and seamless cloud deployment.

Live Demo Flow

Showcase end-to-end interactions with these simple guest login steps:

1. One-Click Login

Click 'One-Click Login as Demo Patient' at /user/login. OTP bypass code 123456 will pre-fill.

2. Upload & Triage

Go to the dashboard, upload a medical report, and check the automatic specialty filter.

3. Confirm Booking

Select a slot. The database lock applies, showing 'Held' payment state.

4. Release Payout

Log in as doctor via dr_smith. Accept appointment and prescribe medicine to trigger doctor payout.